Machine Learning and Data Processing

a.a. 2024-2025

Apprendimento supervisionato per problemi di classificazione

Prof. Ing. F. Benedetto

francesco.benedetto@uniroma3.it
fbenedetto@ieee.org

Chair of "Machine Learning and Data Processing" – University of Roma Tre
Chair of the "Definitions and Concepts for Dynamic Spectrum Access" - IEEE 1900.1

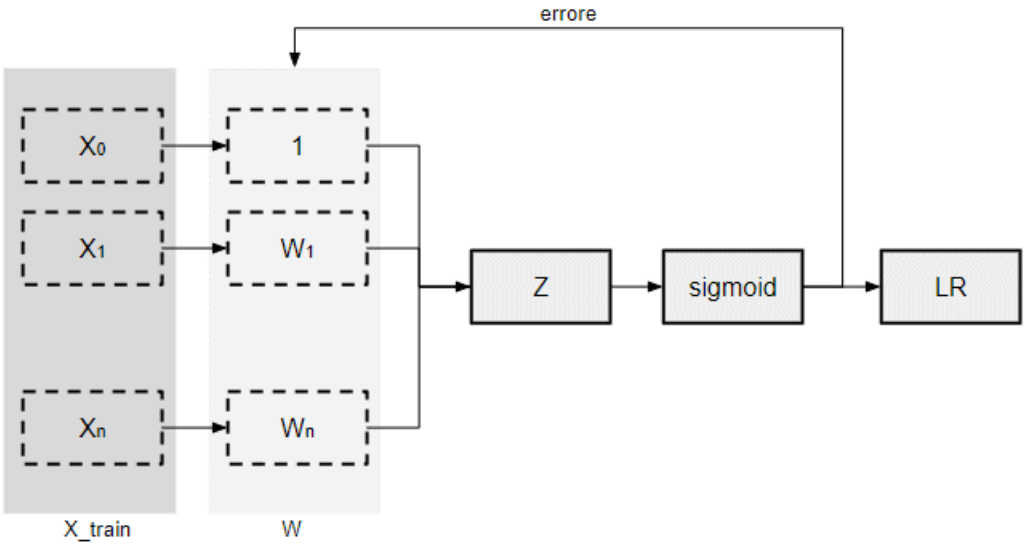
©SP4TE Lab 2017

Regressione logistica

- La regressione logistica è un algoritmo di apprendimento supervisionato che costruisce un modello probabilistico lineare di classificazione dei dati.

- modello di regressione non-lineare utilizzato quando la variabile dipendente è di tipo dicotomico. L'obiettivo del modello è di stabilire la probabilità con cui un'osservazione può generare uno o l'altro valore della variabile dipendente; può inoltre essere utilizzato per classificare le osservazioni.

Nella fase di addestramento l'algoritmo di regressione logistica prende in input n esempi da un **insieme di training (X)**. Ogni singolo esempio è composto da m attributi e dalla classe corretta di appartenenza. Durante l'addestramento l'algoritmo elabora una **distribuzione di pesi (W)** che permetta di classificare correttamente gli esempi con le classi corrette.



Regressione logistica

Poi calcola la combinazione lineare z del vettore dei pesi W e degli attributi x_m

$$z = \vec{x} \cdot \vec{w} + b$$

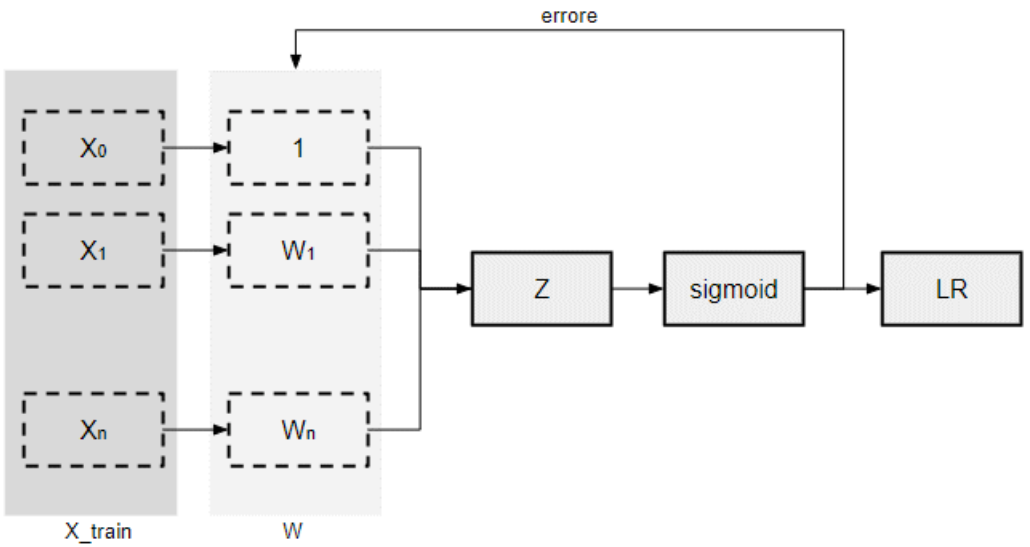
$$z = x_0 \cdot w_0 + \ldots + x_m \cdot w_m + b$$

La combinazione lineare z viene passata alla **funzione logistica (sigmoid)** che calcola la probabilità di appartenenza del campione alle classi del modello.

$$\varphi(z) = \frac{1}{1 + e^{-z}}$$

La funzione sigmoid calcola la probabilità di appartenenza del campione alle classi. La funzione logistica è la funzione di attivazione nei nodi della rete neurale. Il valore della funzione logistica viene convertito in un valore binario (1=attivo 0=non attivo) tramite una funzione di passo unitario (o quantizzatore).

$$\varphi(z = 2.5) = 0.8$$



Ad esempio, se il valore della funzione è maggiore di 0.5 il nodo viene attivato. A questo punto l'algoritmo deve trovare il vettore dei pesi W* e il valore b* in grado di minimizzare l'errore. Questo ricorda quello che accade nella regressione lineare wx+b dove la combinazione w e b deve minimizzare lo scarto quadratico medio.

Regressione logistica

Nella regressione logistica, invece di minimizzare la perdita al quadrato, si massimizza la probabilità secondo la funzione statistica di verosimiglianza al modello.

$$L_{w,b} = \prod_{i=1..N} \varphi(z_{(i)})^{y_i}(1-\varphi(z_{(i)}))^{1-y_i}$$

Dove è la probabilità dell'esempio di appartenere alla classe, considerando che il modello è composto da due sole classi: una classe positiva (es. si), una classe negativa (es. no).

Nota. Si usa l'operatore della produttoria anziché la sommatoria perché la probabilità di osservare N etichette in N esempi è il prodotto della probabilità di ogni singola osservazione. In questa formula si suppone che le probabilità siano indipendenti tra loro.

In alternativa, piuttosto che massimizzare la probabilità, posso massimizzare la verosimiglianza tramite la log-probabilità (entropia binaria incrociata):

$$L_{w,b} = \sum_{i=1}^{N} y_i \log \varphi(z_{(i)})(x) + (1-y_i)\log(1-\varphi(z_{(i)}))$$

Essendo una sommatoria è più agevole da calcolare. Il risultato è equivalente.

Metriche per classificazione

Quando vogliamo analizzare la performance di un classificatore binario, la metrica più comune e accessibile è sicuramente la accuratezza (in inglese *accuracy*). **Essa non fa altro che indicarci quante volte il nostro modello ha correttamente classificato un item nel nostro dataset rispetto al totale.**
Infatti la formula per l'accuratezza è la divisione tra numero di risposte corrette e il totale delle risposte:

accuratezza=numero_risposte_corrette / numero_risposte_totali

L'accuratezza tiene conto delle performance del modello *in senso stretto*. Vale a dire che **non ci permette di comprendere il *contesto* nel quale stiamo operando**. Presa fuori dal contesto, l'accuratezza è una metrica molto delicata da interpretare:

Ad esempio, è sconsigliato usare la accuratezza come metrica di valutazione quando operiamo **con un dataset sbilanciato, dove le classi sono distribuite in maniera impari.** Se la accuratezza non è altro che il rapporto tra risposte corrette sul totale, allora capirete che se una classe compone il 90% del nostro dataset e il nostro modello (erroneamente) classifica ogni esempio nel dataset con quella classe specifica, allora la sua accuratezza sarà del 90%.
Se non siamo attenti potremmo pensare che il nostro modello sia molto performante, quando in realtà è molto lontano dall'esserlo.
La accuratezza è però una metrica sensata da usare se siamo sicuri che il nostro dataset sia bilanciato e che i dati siano di alta qualità.

Matrice di confusione

Per creare una confusion matrix non dobbiamo far altro che elencare le classi effettive presenti nel dataset su righe e quelle che il modello deve prevedere nelle colonne. Il valore delle celle corrisponde alla risposta del modello sotto quelle condizioni.

		Predicted	
		Negative (**N**) -	Positive (**P**) +
Actual	Negative -	True Negatives (T**N**)	False Positives (F**P**) **Type I error**
	Positive +	False Negatives (F**N**) **Type II error**	True Positives (T**P**)

I valori sulla diagonale indicano i punti in cui la predizione del modello di classificazione corrisponde alla classe reale presente nel dataset.

Più elementi sono presenti sulla diagonale, più le nostre predizioni sono corrette

Matrice di confusione

Vediamo una serie di etichette: *true negatives* (veri negativi), *true positives* (veri positivi), *false negatives* (falsi negativi) e *false positives* (falsi positivi). Vediamone uno ad uno.

- **Veri negativi (TN)**: contiene gli esempi che sono stati correttamente classificati come negativi.
- **Veri positivi (TP)**: contiene gli esempi che sono stati correttamente classificati come positivi.
- **Falsi negativi (FN)**: contiene gli esempi che sono stati erroneamente classificati come negativi e che quindi sono in realtà positivi
- **Falsi positivi (FP)**: contiene gli esempi che sono stati erroneamente classificati come positivi e che quindi sono in realtà negativi

		Predicted	
		Negative (**N**) -	Positive (**P**) +
Actual	Negative -	True Negatives (TN)	False Positives (FP) **Type I error**
	Positive +	False Negatives (FN) **Type II error**	True Positives (TP)

Usando una matrice di confusione riusciamo quindi a capire meglio il comportamento del nostro classificatore e come migliorarlo ulteriormente. Per continuare, vediamo come riscrivere la formula della accuratezza dalla matrice di confusione

$$accuratezza=(TN+TP)/(FP+FN+TN+TP)$$

Questa non è altro che il numero di risposte corrette diviso il totale.

Precisione e Richiamo

Definiamo ora la precisione con la formula: *precisione=TP/(FP+TP)*

Come si interpreta? La precisione non è altro che *la accuratezza calcolata solo con classi positive*. Essa è anche chiamata *specificità* poiché definisce quanto sensibile è uno strumento *quando c'è il segnale da riconoscere.* Di fatto, **la metrica ci informa quanto spesso siamo corretti quando classifichiamo una classe come positiva.**

Il **richiamo** invece rappresenta l'altro ago della bilancia. Se siamo interessati a riconoscere quanti più classi positive possibili, allora il nostro modello dovrà avere un richiamo alto.
La sua formula è: r*ichiamo=TP/(FN+TP)*
In pratica significa che qui dobbiamo tener conto dei falsi negativi invece dei falsi positivi. Il richiamo viene anche chiamato *sensibilità* perché all'aumentare del richiamo, il nostro modello diventa sempre meno preciso e classifica anche classi negative come positive.

Un modello ad alta precisione è conservativo: non riconosce sempre la classe correttamente, ma quando lo fa, possiamo stare molto tranquilli che la sua risposta sia corretta.

Un modello ad alto richiamo è liberale: riconosce una classe molto più spesso, ma nel farlo tende a includere anche molto rumore (falsi positivi).

F1 Score

A questo punto è chiaro che usare precisione o richiamo come metrica di valutazione è difficile perché possiamo solo usarne una a scapito dell'altra. Il punteggio F1 risolve proprio questo problema.

Infatti, **il punteggio F1 combina precisione e richiamo in una sola metrica:**
$F1=2\times[(precisione x richiamo)/(precisione+richiamo)]$

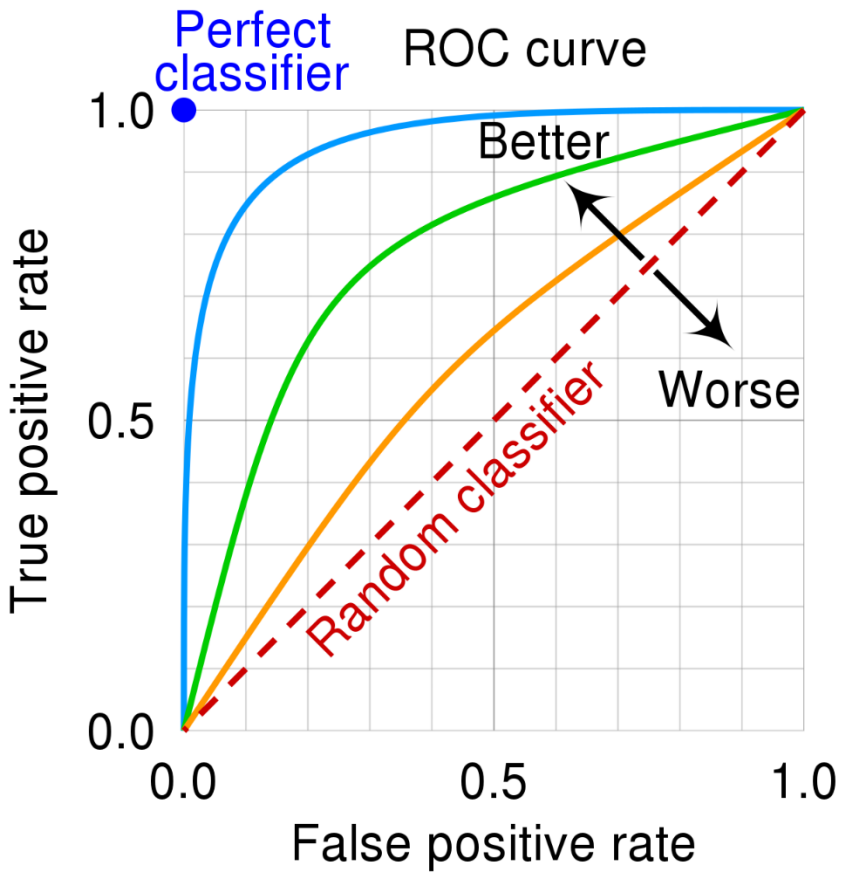
Questa è la media armonica di precisione e richiamo, ed è probabilmente la metrica più usata per valutare modelli di classificazione binaria.

Se il nostro punteggio F1 aumenta, vuol dire che il nostro modello ha aumentato le performance per precisione, richiamo o per entrambi.

ROC e AUC

Le curve ROC sono degli schemi grafici per un classificatore binario. Lungo i due assi si possono rappresentare la sensibilità e (1-specificità), rispettivamente rappresentati da True Positive Rate (TPR, frazione di veri positivi) e False Positive Rate (FPR, frazione di falsi positivi). In altre parole, si studiano i rapporti fra allarmi veri (hit rate) e falsi allarmi.

AUC sta per *Area Under the Curve* (area al di sotto della curva). Se poniamo l'attenzione sulla linea blu, vediamo che al di sotto di essa c'è di fatto una area più grande rispetto alle linee verde arancione. La linea tratteggiata indica un metrica di ROC-AUC del 50%.
Di conseguenza un buon modello avrà una ROC-AUC grande, mentre un modello scarso si posizionerà vicino alla linea tratteggiata, che non è altro che un modello che risponde in maniera casuale.
La metrica ROC-AUC è anche molto utile per confrontare diversi modelli uno contro l'altro.

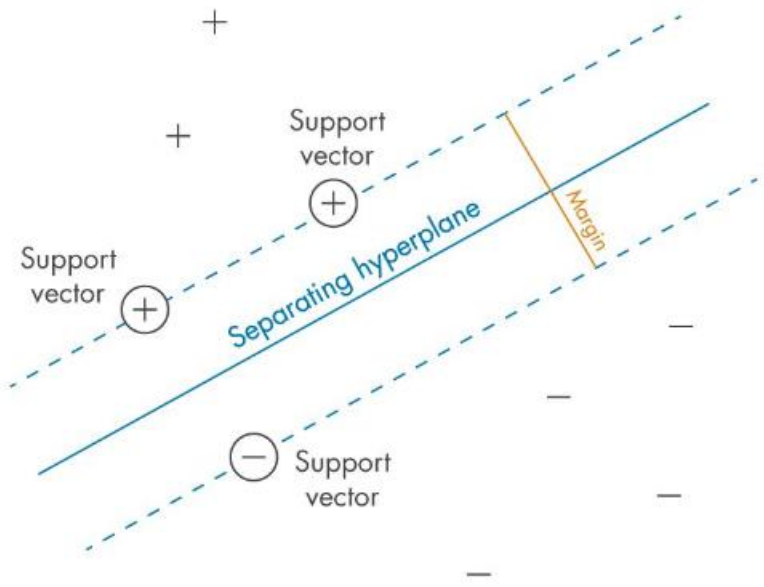


Algoritmi per l'apprendimento supervisionato

Iperpiani di separazione

Supponiamo di dover sviluppare un sistema in grado di classificare se un oggetto appartiene alla classe 1 o alla classe 2 basandosi su 2 attributi (es. identificare il sesso di una persona, conoscendone peso e altezza). Il problema è una classificazione binaria in cui la variabile di input ha solo 2 attributi, ovvero n = 2. Il dataset D è formato da m coppie input/output. Se i dati sono linearmente separabili, allora esisterà sempre un iperpiano di dimensione n-1 in grado di separare le due classi.

Se le m istanze sono linearmente separabili, allora esistono infiniti iperpiani di separazione, ovvero esistono infiniti modelli di classificazione in grado di risolvere il problema senza errori nella fase di training. Per ottenere una soluzione unica, l'idea è quella di scegliere l'iperpiano che massimizza il margine (ossia la distanza) tra i punti appartenenti a una classe e i punti appartenenti all'altra. In questo modo, si minimizza l'errore nella fase di validazione e/o di test.

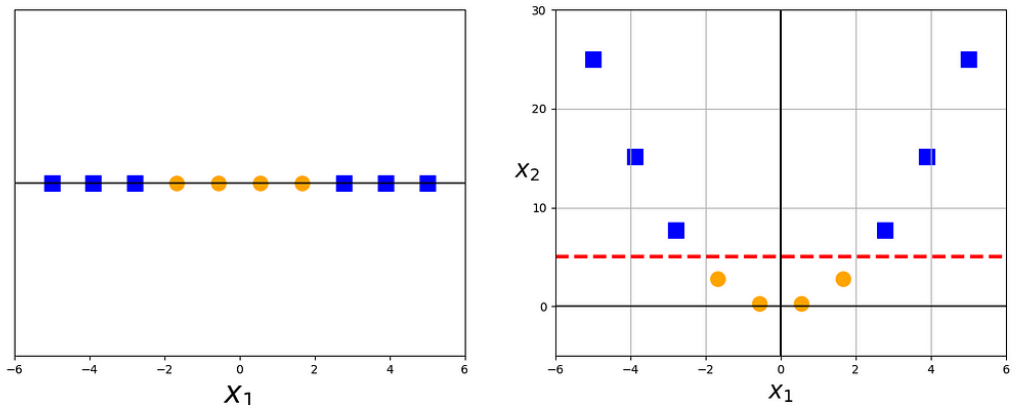


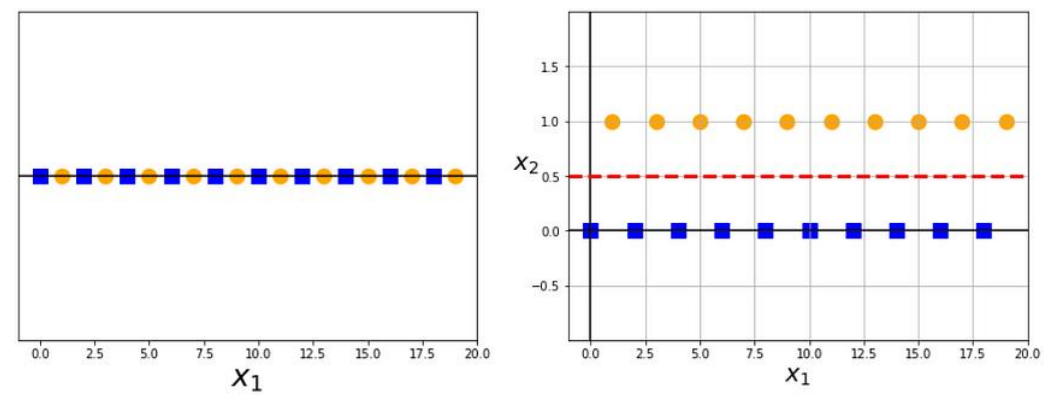
I vettori di supporto determinano l'iperpiano di massimo margine.

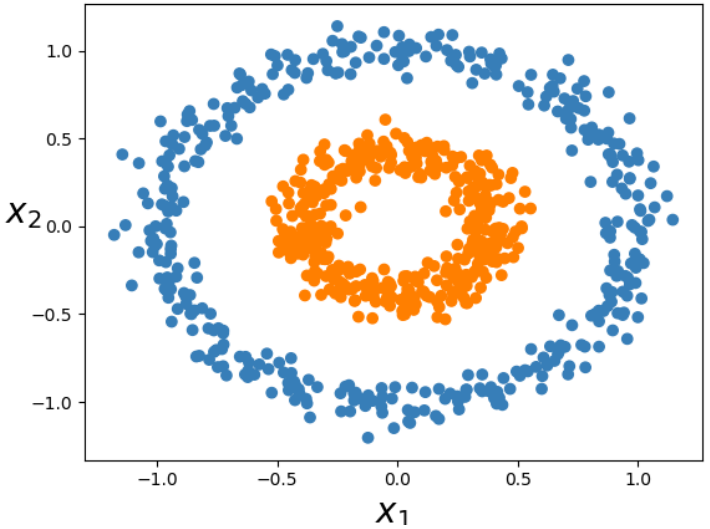
SVM e dati non linearmente separabili

Alcuni problemi sono intrinsecamente non separabili nello spazio delle feature. Tuttavia, dalla conoscenza del problema, è possibile intuire che una trasformazione non lineare possa trasformare lo spazio delle feature di input nello spazio delle feature di output dove l'iperpiano di separazione permette di discriminare meglio le categorie.
Solitamente lo spazio di output ha una dimensione maggiore dello spazio di input. Ciò provoca un aumento della complessità computazionale e di richiesta di risorse.

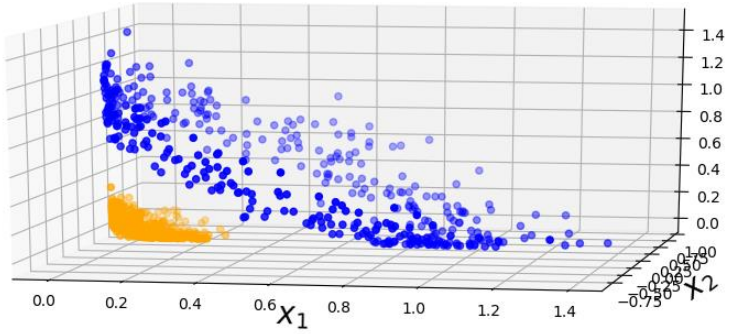
SVM e dati non linearmente separabili







$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} {x_1}^2 \\ \sqrt{2}\, x_1 x_2 \\ {x_2}^2 \end{pmatrix}$$



Kernel Trick

Le Support Vector Machine rientrano in una classe di algoritmi di Machine Learning definiti metodi kernel, in cui è possibile trasformare le feature per mezzo di una funzione kernel. Le funzioni kernel mappano i dati su uno spazio diverso e spesso di dimensione superiore con l'idea che, dopo questa trasformazione, le classi saranno più facili da separare, semplificando potenzialmente i confini decisionali non lineari complessi in confini lineari nello spazio di dimensione superiore delle feature mappate. In questo processo, non è necessario trasformare esplicitamente i dati, il che sarebbe molto dispendioso a livello computazionale. Questo è tradizionalmente noto come il "kernel trick".

I metodi kernel non calcolano le coordinate dei dati nello spazio, ma piuttosto calcolano il prodotto interno tra le immagini di tutte le coppie di dati nello spazio funzione. Quest'operazione è spesso computazionalmente più economica che l'esplicito calcolo delle coordinate, e viene chiamata "stratagemma del kernel" (kernel trick).

Tipo di SVM	Kernel di Mercer	Descrizione
Radial Basis Function (RBF) o gaussiano	$K(x_1, x_2) = \exp\left(-\frac{\lVert x_1 - x_2 \rVert^2}{2\sigma^2}\right)$	Addestramento di una classe. σ è la larghezza del kernel
Lineare	$K(x_1, x_2) = x_1^\mathsf{T} x_2$	Addestramento di due classi.
Polinomiale	$K(x_1, x_2) = \left(x_1^\mathsf{T} x_2 + 1\right)^\rho$	ρ è l'ordine del polinomio
Sigmoide	$K(x_1, x_2) = \tanh\left(\beta_0 x_1^\mathsf{T} x_2 + \beta_1\right)$	Si tratta di un kernel di Mercer solo per alcuni valori β_0 e β_1

Addestrare una Support Vector Machine significa risolvere un problema di ottimizzazione per eseguire il fitting di un iperpiano in grado di minimizzare il margine soft tra le classi. Il numero di feature trasformate è determinato dal numero di vettori di supporto.

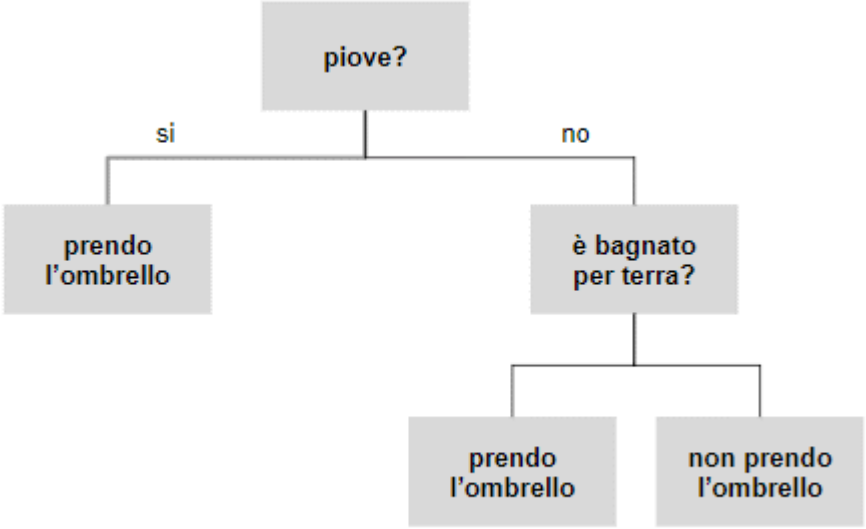
Alberi decisionali

Gli alberi di decisione (o alberi decisionali) sono una rappresentazione dell'apprendimento nel machine learning. Sono anche conosciuti come *decision tree*. In particolar modo, gli alberi di decisione sono utilizzati nei processi di apprendimento induttivo, quelli basati sull'osservazione dell'ambiente circostante.

Un albero di decisione è un sistema con *n* variabili in input e *m* variabili in output. Le variabili in input (attributi) sono derivate dall'osservazione dell'ambiente. Le ultime variabili in output, invece, identificano la decisione / azione da intraprendere. Negli alberi decisionali molto profondi le variabili in output intermedie, in uscita dai nodi genitori, coincidono con le variabili in input dei nodi figli. Le variabili in output intermedie condizionano il percorso verso la decisione finale. Il processo decisionale è rappresentato con un albero logico rovesciato dove ogni nodo è una funzione condizionale:

Ogni nodo verifica una condizione (test) su una particolare proprietà dell'ambiente (variabile) e ha due o più diramazioni verso il basso.

Il processo consiste in una sequenza di test. Comincia sempre dal nodo radice, il nodo genitore situato più in alto nella struttura, e procede verso il basso.



WWW.ANDREAMININI.COM

Alberi decisionali

A seconda dei valori rilevati in ciascun nodo, il flusso prende una direzione oppure un'altra e procede progressivamente verso il basso.

Man mano che il processo di selezione prosegue verso il basso, lo spazio delle ipotesi si riduce perché gran parte dei rami decisionali dell'albero sono eliminati. La decisione finale si trova nei nodi foglia terminali, quelli più in basso.

In questo modo, dopo aver analizzato le varie condizioni, l'agente giunge alla decisione finale.

In un albero decisionale le variabili possono essere:

Variabili discrete. Le variabili hanno valori numerici interi.
In questi casi si parla di classificazione. *Nota*: La forma discreta più semplice è la classificazione booleana dove le variabili hanno soltanto due valori: zero (falso) o uno (vero).

Variabili continue. Le variabili hanno valori numerici reali.
Dati due numeri reali qualsiasi, c'è sempre un altro numero intermedio tra questi.
In questi casi si parla di regressione. *Nota:* La rappresentazione delle variabili continue è molto più complessa ma si adatta meglio alla logica sfumata (fuzzy logic) e alla logica in condizioni di incertezza, quando non esiste una distinzione netta tra il vero e il falso, tra un'affermazione e la sua negazione, ecc.

Alberi decisionali

Vantaggi e svantaggi:

I vantaggi - Gli alberi logici hanno l'indiscusso vantaggio della semplicità. Sono facili da capire e da eseguire. Rispetto alle reti neurali l'albero decisionale è facilmente comprensibile dagli esseri umani. Pertanto, l'uomo può verificare come la macchina giunge alla decisione. Eventualmente dissentire. *Esempio*: Un albero decisionale applicato alla medicina fornisce delle diagnosi. Essendo una decisione importante per il paziente, è sempre opportuno che un medico verifichi il processo di classificazione che ha portato la macchina a prendere quella decisione. Per un uomo è senza dubbio più facile farlo leggendo un albero decisionale che una rete neurale. Inoltre, gli alberi decisionali booleani sono facilmente sviluppabili sotto forma di codice di programmazione, perché possono essere rappresentati con qualsiasi linguaggio proposizionale. Tutte le funzioni booleane possono essere rappresentate come albero di decisionale. E viceversa. Questa caratteristica è detta *espressività* degli alberi decisionali.

Gli svantaggi - La rappresentazione ad albero decisionale è poco adatta per i problemi complessi, perché lo spazio delle ipotesi diventa troppo grande. La complessità spaziale dell'algoritmo potrebbe diventare proibitiva. Un albero decisionale può descrivere soltanto una relazione tra una funzione di verità e le combinazioni logiche di attributi. Non riesce a rappresentare più funzioni. Per farlo occorre creare un altro albero decisionale e associarlo al precedente. Ogni funzione di verità è un albero decisionale a se stante. Inoltre, l'albero decisionale non riesce a rappresentare tutti i tipi di funzioni ma soltanto alcuni. *Esempio*. L'albero di decisione non riesce a rappresentare una funzione che restituisce 1 quando tutti gli attributi sono veri (funzione di parità) o quando la maggioranza degli attributi sono veri (funzioni di maggioranza).

Alberi decisionali

Criteri di scelta degli attributi:

Dobbiamo capire come scegliere gli attributi secondo cui suddividere il dataset. Limitando l'osservazione a due soli attributi, *j* e *v*, potremmo suddividere il dataset attraverso una funzione O come segue:;

$$O(j, v) = (L_s/L) * G_s + (L_d/L) * G_d$$

Dove L_s e L_d sono il numero di elementi ricadenti a sinistra e destra nella divisione del dataset e G_d e G_s sono misure del grado di disordine dei dati a destra e sinistra.

Per costruire l'albero occorre calcolare tutte le possibili combinazioni per ogni nodo e infine scegliere la combinazione per cui il valore di O(j, v) è minimo.

Questo approccio è di tipo euristico, *greedy*, essendo un'ottimizzazione locale a livello di nodo. Il problema complessivo sarebbe altrimenti NP-completo.

Day	Outlook	Temp	Humidity	Wind	Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Weak	Yes

Alberi decisionali

$$\text{Entropy}(S) = -\sum_{c \in C} p(c)\log_2 p(c)$$

$$\text{Information Gain}(S,a) = \text{Entropy}(S) - \sum_{v \in values(a)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

- *a* rappresenta un attributo specifico o un'etichetta di classe
- *Entropy(S)* è l'entropia del set di dati, S
- |Sv|/ |S| rappresenta la proporzione dei valori in S_v per il numero di valori nel set di dati, S
- *Entropy(S_v)* è l'entropia del set di dati, S_v

Per questo set di dati, l'entropia è 0,985. Questa può essere calcolata risalendo alla proporzione dei giorni in cui "Giocare a tennis" è "Sì", che è 4/7, e la proporzione dei giorni in cui "Giocare a tennis" è "No", che è 3/7. Quindi, questi valori possono essere inseriti nella formula dell'entropia di cui sopra.
Entropia (Tennis) = -(4/7) log2(4/7) – (3/7) log2 (3/7) = 0,985
Possiamo quindi calcolare le informazioni di guadagno per ciascuno dei singoli attributi. Ad esempio, il guadagno di informazione per l'attributo "Vento" sarebbe il seguente:
Guadagno (Tennis, Vento) = (0,985)-(5/7)*(0,863) – (2/7)*(0,863) = 0,1219.

Day	Outlook	Temp	Humidity	Wind	Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Weak	Yes

Riepilogando,
- 5/7 rappresenta la proporzione di valori in cui il vento è “forte” rispetto al numero totale di valori di vento
- 0,863 è l'entropia quando vento = «forte»

Quindi, ripetere il calcolo del guadagno di informazione per ciascun attributo nella tabella precedente e selezionare l'attributo con il guadagno di informazioni più elevato come primo punto di divisione nell'albero decisionale. In questo caso, il risultato rappresenta il maggior guadagno di informazioni. Da qui, il processo viene ripetuto per ogni albero secondario.

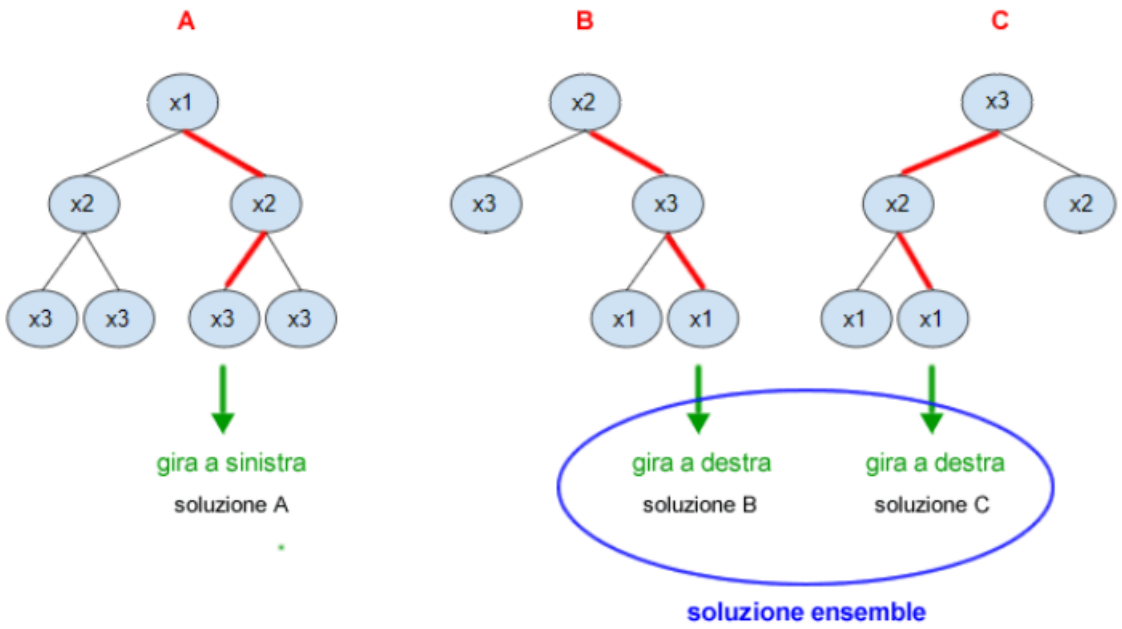
Ensemble learning

Il termine ensemble significa "collezione" e indica un insieme di ipotesi che la macchina elabora. A differenza dei sistemi più semplici, per formulare una previsione la macchina non estrapola una sola ipotesi ma molte.

Nota: Nel machine learning un'ipotesi è un albero decisionale.

Costruendo più decision tree a partire dallo stesso insieme di dati, si riduce la probabilità di errore, perché ogni albero segue un metodo di ragionamento differente. E' quindi probabile che gli alberi forniscano risposte diverse.

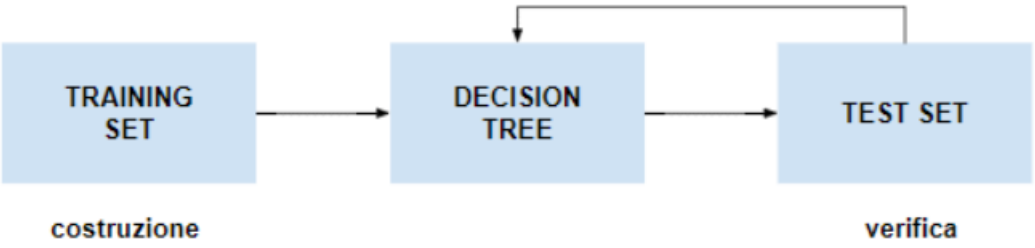
Per prendere una decisione la macchina consulta tutti gli alberi decisionali che ha estrapolato. Poi sceglie la risposta fornita dalla maggioranza degli decision tree.

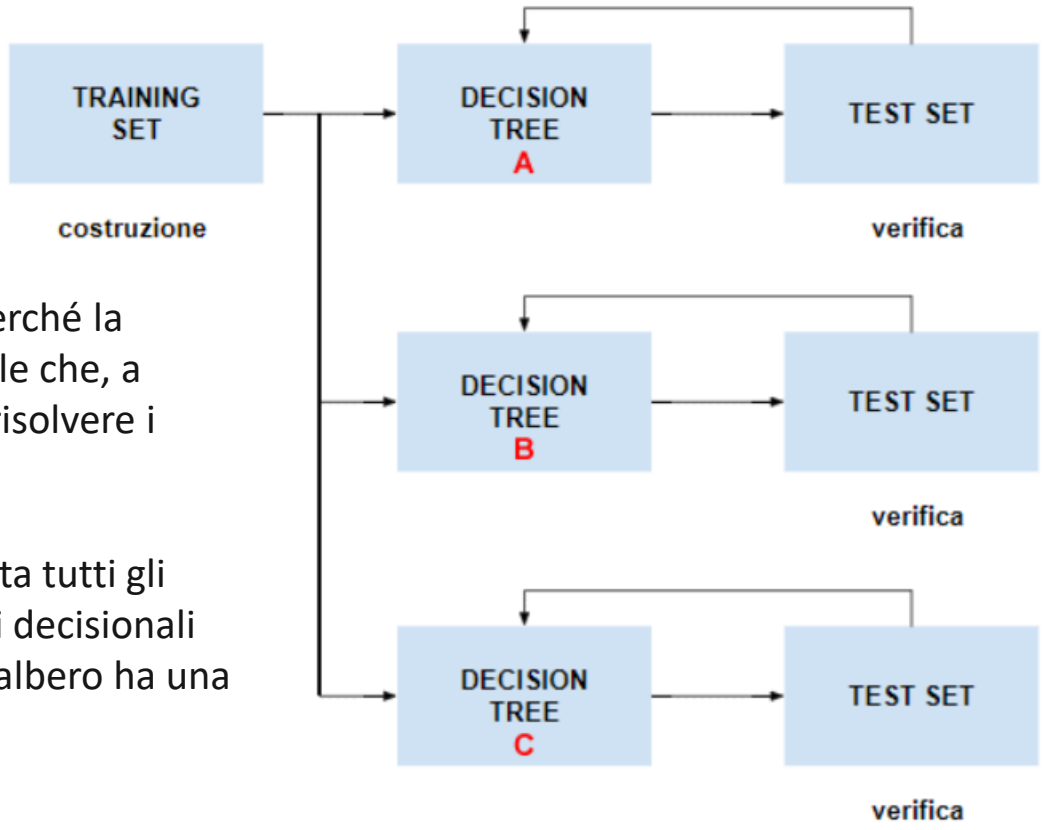


Ensemble learning: esempio

Il processo di ensemble learning è suddiviso in due fasi:

1) La costruzione delle ipotesi: La macchina utilizza un training set (dati di addestramento) e a partire da questo insieme di dati costruisce un albero decisionale. Nel caso dell'ensemble learning, la macchina elabora N alberi decisionali diversi tra loro.





Ogni albero decisionale (A, B, C) si distingue dagli altri per forma, perché la macchina seleziona in modo differente gli attributi. E' quindi probabile che, a parità di condizioni, ogni albero fornisca una risposta differente per risolvere i problemi pratici.

2) La fase decisionale: Per risolvere un problema, la macchina consulta tutti gli alberi decisionali dello spazio delle ipotesi che ha costruito. Gli alberi decisionali forniscono risposte diverse, perché seguono logiche differenti. Ogni albero ha una probabilità *Px* di errore.

La macchina sceglie la soluzione più frequente
o quella fornita dalla maggioranza degli alberi decisionali.

Ensemble learning: decorrelazione di modelli

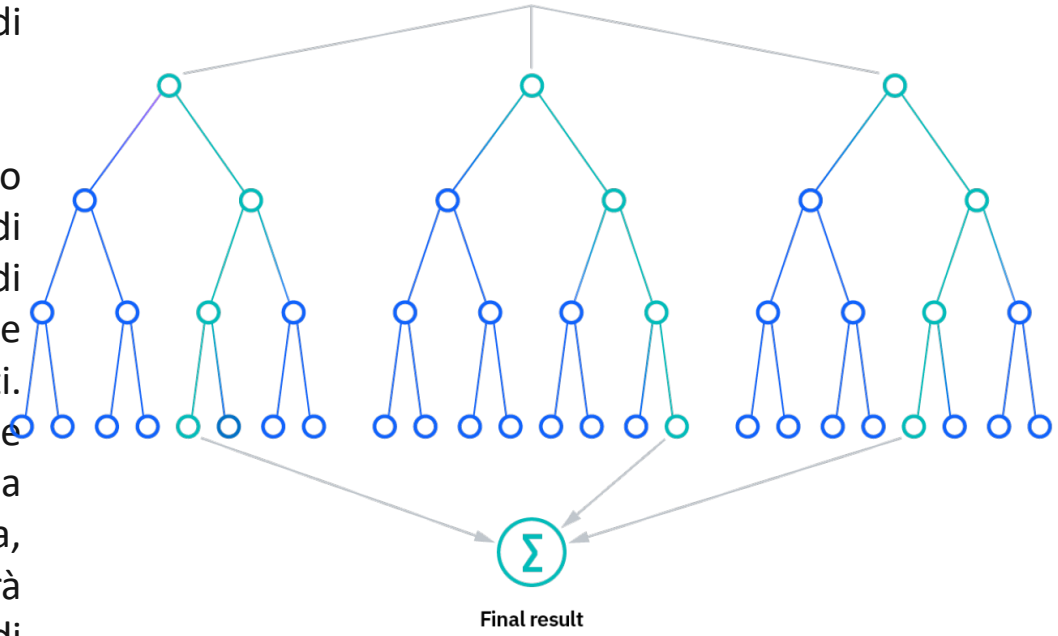
Il problema principale dell'EL è garantire che i modelli siano indipendenti: tanto meno è rispettata la condizione di indipendenza, tanto meno affidabile sarà la stima dell'ensemble.
Per ovviare a ciò si potrebbero utilizzare modelli diversi (es. regressione logistica, SVM, DT). Oppure utilizzare lo stesso modello su subset diversi del set di training. Ciò presuppone un training set esteso, ma spesso non disponibile. Occorre quindi definire delle strategie per utilizzare con parsimonia i pochi dati a disposizione. Una strategia è il *bagging*: in questo metodo, un campione casuale di dati in un set di addestramento viene selezionato con sostituzione, il che significa che i singoli punti di dati possono essere scelti più di una volta. Quindi i sottoinsiemi creati condividono (probabilisticamente) alcune istanze. Ad una prima analisi potrebbe sembrare no particolarmente vantaggioso utilizzare dei dataset con un numero così elevato di duplicati. Tuttavia i duplicati, essendo differenti per ciascun sottoinsieme, consentono la creazione di modelli indipendenti, perché ogni modello tenderà a focalizzare maggiormente l'addestramento sulle istanze che presentano duplicati.

Algoritmo Random Forest: è un'estensione del metodo di bagging in quanto utilizza sia il bagging che la casualità delle caratteristiche per creare una foresta non correlata di strutture ad albero decisionali. La casualità delle caratteristiche, nota anche come bagging delle caratteristiche o «metodo del sottospazio casuale», genera un sottoinsieme casuale di caratteristiche, che garantisce una bassa correlazione tra le strutture ad albero decisionali. Si tratta di una differenza fondamentale tra strutture ad albero decisionali e Random Forest. Mentre le strutture ad albero decisionali considerano tutte le possibili suddivisioni delle caratteristiche, gli algoritmi Random Forest selezionano solo un sottoinsieme di queste caratteristiche.

Random Forest

Gli algoritmi Random Forest hanno tre iper-parametri principali, che devono essere impostati prima dell'addestramento. Questi iperparametri includono la dimensione dei nodi, il numero di strutture ad albero e il numero di caratteristiche campionate. Da lì, il classificatore Random Forest può essere utilizzato per risolvere problemi di regressione o classificazione.

L'algoritmo Random Forest è costituito da una serie di strutture ad albero decisionali e ogni struttura ad albero è un insieme composto da un campione di dati tratto da un set di addestramento con sostituzione, chiamato esempio di bootstrap. Di questo campione di addestramento, un terzo è messo da parte come dati di test, noto come campione oob (out-of-bag), su cui torneremo più avanti. Un'altra istanza di casualità viene poi inserita attraverso il bagging delle caratteristiche, aggiungendo maggiore diversità al set di dati e riducendo la correlazione tra le strutture ad albero decisionali. A seconda del tipo di problema, la determinazione della previsione potrà variare. Per un'attività di regressione, sarà calcolata la media delle singole strutture ad albero decisionali, e per un attività di classificazione, un voto di maggioranza - ossia la variabile categoriale più frequente - produrrà la classe prevista. Infine, il campione oob viene poi utilizzato per la convalida incrociata, finalizzando tale previsione.



Random Forest

Vantaggi:
- Rischio di overfitting ridotto: le strutture ad albero decisionali corrono il rischio di un overfitting in quanto tendono ad adattarsi strettamente a tutti i campioni all'interno dei dati di addestramento. Tuttavia, quando esiste un numero consistente di strutture ad albero decisionali in un Random Forest, il classificatore non si adatta eccessivamente al modello poiché la media delle strutture ad albero non correlate riduce la varianza complessiva e l'errore di previsione.

- Fornisce flessibilità: dal momento che Random Forest può gestire sia le attività di regressione che di classificazione con un alto grado di accuratezza, è un metodo diffuso tra i data scientist. Il bagging delle caratteristiche rende inoltre il classificatore Random Forest uno strumento efficace per stimare i valori mancanti, in quanto mantiene l'accuratezza quando una parte dei dati risulta mancante.

Svantaggi:
- Processo che richiede tempo: dal momento che gli algoritmi Random Forest possono gestire grandi set di dati, possono fornire previsioni più accurate, ma possono essere lenti a elaborare i dati perché calcolano i dati per ogni singola struttura ad albero decisionale.

- Richiede più risorse: poiché gli algoritmi Random Forest elaborano grandi set di dati, richiedono più risorse per l'archiviazione di tali dati.

- Più complesso: la previsione di una singola struttura ad albero decisionale è più facile da interpretare rispetto a una loro foresta.

Ensemble in sequenza: Boosting

Nel boosting ogni modello viene costruito in modo sequenziale in base agli errori del modello precedente. Dei dati vengono forniti al primo classificatore, il quale effettua una prediction. Nel classificatore successivo viene dato più peso alle istanze classificate in modo sbagliate. In questo modo l'enfasi viene posta sugli errori dei classificatori precedenti. Alla base di questo algoritmo sta il fatto di formare un certo numero di classificatori deboli che, imparando dagli errori dei precedenti e in molte iterazioni, si trasformano in un classificatore forte.

Supponendo di aver definito una funzione di errore L, si addestra un primo modello K_1, e per ogni istanza i si calcola l'errore residuo del modello $e^i(K_1)$:

$$e^i(K_1) = y^i - K_1(x^i)$$

Alla fine si otterranno m valori residui per l'errore residuo, uno per ogni istanza del training set. Se ognuno di questi valori è pari a zero, allora K_1 è un sistema perfetto e non occorre andare oltre. Tuttavia, nei casi pratici, esisteranno degli errori. Tali errori vengono quindi utilizzati come variabili dipendenti per addestrare un nuovo modello K_2, per minimizzare l'errore residuo $e^i(K_1)$, ottenendo un uovo errore residuo $e^i(K_2)$:

$$e^i(K_2) = e^i(K_1) - K_2(x^i)$$

La procedura viene iterata finché non interviene un criterio di arresto. Alla fine il modello ensemble ottenuto non sarà altro che la somma dei diversi modelli addestrati in sequenza:

$$K_1(x^i) + K_2(x^i) + K_3(x^i) + \ldots = \hat{y}^i$$